Lab 4

[valid 2024-2025]

Starting from this week...:

- Use appropriate collections in order to represent data (otherwise: -0.5 points)
- Use Java Stream API in order to perform aggregate operations on data.

Collections, Java Stream API
A robot (drone) must explore certain locations on a map, each location having a distinct name and a type: friendly, neutral or enemy.
For every pair (a,b) of locations, the following information is known: if the robot can move directly from a to b, the time needed to go from a to b and the probability to reach b from a safely (directly means without going through other locations).
Initially, the robot is at a start location. Then, it receives commands to go from its current location to another, either in the shortest time or on the safest route.
The problem is to instruct the robot on which paths to go in order to execute its commands (a path is a sequence of locations).

The main specifications of the application are:

Compulsory (1p)

- Create a Maven project.
- Create an object-oriented model of the problem.
- Create an array of locations, of various types.
- Using Stream API, put all the friendly locations in a *TreeSet* and print them sorted by their natural order.
- Using Stream API, put all the enemy locations in a *LinkedList* and print them sorted by their type and then by their name.

Homework (2p)

- Use a third-party library in order to generate random fake names for locations.
- Use one of the following libraries in order to determine the fastest routes from the start location to all other locations:
  - Graph4J
  - JGraphT
- Using Stream API, create a data structure that stores, for each type, the locations of that type.
- Display the fastest times computed above: first for the friendly locations, then for the neutral and then for the enemy locations.

Bonus (2p)

- Determine the safest routes for any pair of locations. Create a data structure that stores, for each computed route, the number of locations of each type.
- Create a random problem generator and run the algorithms for large instances, having a number of locations ranging from hundreds to thousands.
- Store the results of your tests in a data structure and compute various statistics, using Java Stream API.

Resources

- Slides
- Aggregate Operations
- The Java Tutorials: Collections (Know Thy Complexities!)
- Setting the class path
- Creating, Importing, and Configuring Java Projects in Netbeans
- Packaging Programs in JAR Files

Objectives

- Use various collections and polymorphic algorithms.
- Use Java Stream API.
- Use *Optional* class and *var* keyword.
- Understand the notion of CLASSPATH.
- Use external JAR libraries.
- Create a Maven Project.
- Package all project classes as an executable JAR file.